

Módulo de Calendario Astrológico y Efemérides – Arquitectura y Plan Codex

****Proyecto:**** AstroMalik-macOS

****Autor:**** Claude Opus 4.6 (Anthropic) – rol: Arquitecto SW + Astrólogo

****Fecha:**** 4 de mayo de 2026

****Alcance:**** Diseño completo del módulo Calendario/Efemérides, especificación funcional astrológica, arquitectura Swift, modelos, integración con la app existente y prompt de implementación para Codex.

1. Visión del Módulo

1.1 Qué es y por qué importa

Un ****Calendario Astrológico**** no es un listado plano de posiciones planetarias. Es la herramienta que responde a la pregunta diaria del astrólogo:

> ****¿Qué está pasando en el cielo hoy, esta semana, este mes?****

Mientras que el módulo de Tránsitos responde "¿qué le pasa al cielo *respecto a mi carta*", el Calendario responde "¿qué le pasa *al cielo en sí mismo*". Son complementarios: el Calendario muestra el clima cósmico general; los Tránsitos lo personalizan.

1.2 Diferencia con los Tránsitos existentes

Dimensión	Tránsitos (actual)	Calendario (nuevo)
---	---	---
Referencia	Carta natal del usuario	El cielo en sí mismo
Pregunta	"¿Qué me afecta?"	"¿Qué pasa en el mundo?"
Cuerpos	Planetas sobre puntos natales	Todos los planetas entre sí + Luna
Eventos	Aspectos a natal, ingresos por casa natal	Lunaciones, eclipses, estaciones, VoC, ingresos por signo, aspectos mundanos
Periodo típico	6 meses–2 años	1 día–1 mes vista
Granularidad	Evento con rango de fechas	Día a día, hora a hora (para Luna)

1.3 Valor comercial

Según el análisis comercial del proyecto, un calendario astrológico es prioridad ****alta**** para justificar el precio de 52–79 €. Es una funcionalidad que las apps de escritorio de pago ofrecen de serie (TimePassages, Astro Gold, Solar Fire, Mastro). Las alternativas gratuitas como Planetdance y Astrolog también lo incluyen. Su ausencia es un gap visible.

2. Funcionalidades Astrológicas – Especificación Doctrinal

2.1 Lunaciones (Luna Nueva y Luna Llena)

****Qué son:**** Los momentos exactos en que Sol y Luna forman conjunción (Luna Nueva) u oposición (Luna Llena).

****Por qué importan:**** Son los marcadores de ritmo más básicos de la astrología. Todo ciclo de 28-29 días arranca con una Luna Nueva (siembra) y culmina con una Luna Llena (cosecha/revelación). Un astrólogo profesional siempre sabe cuándo son las próximas lunaciones y en qué signo/grado caen.

****Datos a calcular:****

- Fecha y hora exacta (UTC y local) de cada Luna Nueva y Luna Llena.
- Longitud eclíptica del evento (grado, signo).
- Casa natal donde cae la lunación (si hay carta activa – opcional, no obligatorio para v1).
- Saros number N0 es necesario en v1.

****Método de cálculo:****

- Buscar los momentos en que $|long_Sol - long_Luna| == 0^\circ$ (Luna Nueva) o $== 180^\circ$ (Luna Llena).
- Swiss Ephemeris ofrece ``swe_sol_eclipse_when_glob`` para eclipses solares, pero para lunaciones genéricas es más limpio hacer una búsqueda iterativa por bisección del ángulo Sol-Luna sobre el rango de fechas. Se calcula la diferencia angular Sol-Luna cada 6 horas, se detectan cruces de 0° y 180° , y se refina con bisección a precisión de ~1 minuto.
- Alternativa más directa: usar ``swe_mooncross_ut`` para encontrar el momento en que la Luna alcanza la longitud del Sol (Luna Nueva). Para Luna Llena, buscar cuándo la Luna alcanza ``long_Sol + 180``.

2.2 Eclipses

****Qué son:**** Lunaciones que ocurren cerca del eje nodal, produciendo ocultación real del Sol (eclipse solar = Luna Nueva cerca de los Nodos) o de la Luna (eclipse lunar = Luna Llena cerca de los Nodos).

****Por qué importan:**** Los eclipses son las lunaciones más potentes. En astrología clásica y moderna, un eclipse marca un punto de inflexión que se activa cuando un tránsito posterior toca el grado del eclipse. Un eclipse en tu casa 7 o sobre tu Sol natal es un evento de primer orden. Muchos astrólogos profesionales planifican el año entero alrededor de los eclipses.

****Datos a calcular:****

- Tipo: solar total/anular/parcial, lunar total/parcial/penumbral.
- Fecha y hora exacta.
- Longitud eclíptica (grado, signo).
- Magnitud (para evaluar potencia).
- Visibilidad local (si el eclipse es visible desde la ubicación del usuario – interesante pero no imprescindible v1).
- Nodo asociado (Norte o Sur) y distancia angular al Nodo.

****Método de cálculo:****

- ``swe_sol_eclipse_when_glob()`` – busca el siguiente eclipse solar global desde un JD dado. Devuelve array ``tret[]`` con tiempos de máximo, inicio y fin. Devuelve flags de tipo (total, anular, parcial).
- ``swe_lun_eclipse_when()`` – busca el siguiente eclipse lunar global. Devuelve tipo y tiempos.
- ``swe_sol_eclipse_how()`` / ``swe_lun_eclipse_how()`` – para un JD y ubicación dados, devuelve atributos como magnitud.
- Para listar eclipses de un año: bucle llamando ``swe_sol_eclipse_when_glob`` y ``swe_lun_eclipse_when`` avanzando desde cada resultado hasta cubrir el rango.

2.3 Estaciones Planetarias (Retrogradaciones)

****Qué son:**** Los momentos en que un planeta cambia de dirección aparente: de directo a retrógrado (estación retrógrada, SR) o de retrógrado a directo (estación directa, SD).

****Por qué importan:**** Una estación planetaria es un momento de máxima intensidad del planeta. Mercurio retrógrado es famoso incluso fuera de la astrología, pero las estaciones de Saturno, Júpiter, Marte y Venus son técnicamente más importantes. Cuando un planeta se estaciona, su velocidad eclíptica es $\sim 0^\circ/\text{día}$, y su influencia se concentra en un grado concreto durante semanas. Si ese grado toca un punto natal, es extremadamente potente.

****Datos a calcular:****

- Planeta.
- Tipo de estación: SR (retrógrada) o SD (directa).
- Fecha y hora exacta.
- Longitud eclíptica (grado, signo).
- Velocidad diaria en el momento de la estación (será ~ 0 , confirmación de exactitud).

****Método de cálculo:****

- Para cada planeta (Mercurio a Plutón, no Sol ni Luna que nunca retrogradan):
 - Calcular velocidad eclíptica (``xx[3]`` del ``swe_calc_ut`` con ``SEFLG_SPEED``) cada día del rango.
 - Detectar cambios de signo en la velocidad (positiva $\rightarrow$ negativa = estación retrógrada; negativa $\rightarrow$ positiva = estación directa).
 - Refinar por bisección el JD exacto donde la velocidad cruza cero.
- No se necesita función especial de Swiss Ephemeris para esto; es detección de cruce de cero en la derivada longitudinal.

****Planetas relevantes:****

- Mercurio: 3 retrogradaciones/año (~ 3 semanas cada una).
- Venus: 1 cada ~ 18 meses (~ 40 días).
- Marte: 1 cada ~ 2 años (~ 2.5 meses).
- Júpiter: 1 al año (~ 4 meses retrógrado).
- Saturno: 1 al año (~ 4.5 meses retrógrado).

- Urano: 1 al año (~5 meses retrógrado).
- Neptuno: 1 al año (~5.5 meses retrógrado).
- Plutón: 1 al año (~5-6 meses retrógrado).

2.4 Ingresos en Signos

****Qué son:**** El momento en que un planeta cruza de un signo al siguiente (cruza un múltiplo de 30°).

****Por qué importan:**** Un ingreso marca un cambio de tono colectivo. Saturno entrando en Aries cambia el tono de responsabilidad/restricción durante 2.5 años. Júpiter entrando en Géminis cambia la zona de expansión. Incluso los ingresos del Sol (que definen las estaciones astronómicas: Aries = equinoccio de primavera) son significativos para la astrología mundana.

****Datos a calcular:****

- Planeta.
- Signo al que ingresa.
- Fecha y hora exacta.
- Si es ingreso directo o retrógrado (un planeta puede "ingresar" en un signo, retroceder al anterior, y volver a ingresar – triple ingreso).
- Dirección: directo o retrógrado en el momento del cruce.

****Método de cálculo:****

- Para cada planeta, calcular la longitud diaria y detectar cruces de múltiplos de 30°.
- `swe\_solcross\_ut` existe para el Sol pero no para otros planetas en la API estándar. Para planetas genéricos, bisección sobre la longitud.
- Caso especial: cuando un planeta está retrógrado, puede cruzar el límite de signo hacia atrás. Hay que detectar esos cruces también.

****Planetas relevantes:****

- Sol: 12 ingresos/año (uno por signo), define las estaciones.
- Luna: ~12-13 ingresos/mes – demasiado frecuente para la vista mensual principal, pero útil en la vista diaria.
- Mercurio a Plutón: según su velocidad.

2.5 Luna Vacía de Curso (Void of Course, VoC)

****Qué es:**** El periodo entre el último aspecto exacto ptolemaico (conjunción, sextil, cuadratura, trígono, oposición) que la Luna forma con otro planeta antes de cambiar de signo, y el momento en que la Luna entra en el signo siguiente.

****Por qué importa:**** Es una de las reglas más usadas en astrología electiva y horaria. "No inicies nada importante cuando la Luna está vacía de curso." Muchos astrólogos profesionales consultan el VoC antes de programar reuniones importantes, firmar contratos o iniciar proyectos. En horaria, el motor nativo de AstroMalik ya detecta VoC como condición de la pregunta – aquí lo llevamos al calendario general.

****Datos a calcular:****

- Inicio del VoC: fecha/hora del último aspecto ptolemaico exacto de la Luna antes de cambiar de signo.
- Fin del VoC: fecha/hora del ingreso de la Luna en el signo siguiente.
- Duración.
- Último aspecto: qué planeta, qué aspecto.
- Signo actual de la Luna durante el VoC.

****Método de cálculo:****

1. Calcular el momento en que la Luna cambia de signo (ingreso lunar).
2. Retroceder desde ese momento y buscar el último aspecto ptolemaico exacto de la Luna con cualquiera de los 9 planetas restantes (Sol a Plutón, opcionalmente solo los 6 tradicionales para versión clásica).
3. El VoC va desde ese último aspecto exacto hasta el ingreso.

****Nota doctrinal:**** Hay dos tradiciones sobre qué planetas cuentan para VoC:

- ****Clásica (Lilly):**** Solo los 7 planetas tradicionales (Sol a Saturno). VoC más largos.
- ****Moderna:**** Los 10 planetas (Sol a Plutón). VoC más cortos porque Urano, Neptuno y Plutón se mueven tan lento que la Luna siempre encuentra un aspecto con ellos relativamente cerca.
- ****Recomendación v1:**** Usar los 10 planetas por defecto con opción futura de filtrar a clásicos.

2.6 Aspectos Diarios del Cielo (Aspectos Mundanos)

****Qué son:**** Los aspectos exactos que se forman entre planetas en tránsito, sin referencia a ninguna carta natal.

****Por qué importan:**** Definen el "clima" astrológico del día. "Hoy Venus trígono Júpiter" es un día de armonía social y abundancia. "Hoy Marte cuadratura Saturno" es un día de frustración y bloqueos. Los astrólogos profesionales consultan estos aspectos para entender el tono general del día.

****Datos a calcular:****

- Planeta A y Planeta B.
- Tipo de aspecto (los 5 ptolemaicos, quincuncio opcional).
- Fecha y hora exacta del aspecto.
- Si alguno de los planetas está retrógrado.
- Calificación básica: benéfico/maléfico/neutro (basada en naturaleza de planetas y aspecto).

****Método de cálculo:****

- Para cada par de planetas, calcular la diferencia angular diaria y detectar el momento exacto en que alcanza 0°, 60°, 90°, 120° o 180° (con orbe = 0° para el momento exacto, pero calculando la ventana de aplicación/separación para el listado diario).
- Solo planetas lentos entre sí (no Sol-Luna ni Luna-planetas en la

vista mensual, porque son demasiado frecuentes). La Luna sí puede incluirse en la vista diaria.

2.7 Fase Lunar Diaria

****Qué es:**** La fase de la Luna en un día dado (nueva, creciente, primer cuarto, gibosa creciente, llena, gibosa menguante, último cuarto, menguante).

****Por qué importa:**** Es la información astrológica más básica y más consultada. La fase lunar aparece en cualquier calendario astrológico, agenda espiritual o app de bienestar. Para el usuario de AstroMalik es un anclaje visual inmediato.

****Datos a calcular:****

- Fase (8 fases principales o al menos 4: nueva, cuarto creciente, llena, cuarto menguante).
- Porcentaje de iluminación (dato que Swiss Ephemeris puede dar con `swe\_pheno\_ut`).
- Signo de la Luna.

****Método de cálculo:****

- Diferencia angular Sol-Luna módulo 360°:
 - 0°: Nueva
 - 0°-90°: Creciente
 - 90°: Primer cuarto
 - 90°-180°: Gibosa creciente
 - 180°: Llena
 - 180°-270°: Gibosa menguante
 - 270°: Último cuarto
 - 270°-360°: Menguante

2.8 Efeméride Diaria

****Qué es:**** Una tabla con las posiciones exactas de todos los planetas para cada día del mes (o rango).

****Por qué importa:**** Es la herramienta de referencia rápida del astrólogo. Antes del software, los astrólogos compraban libros de efemérides. Tener una efeméride integrada evita abrir Astro.com o buscar un PDF.

****Datos a calcular:****

- Para cada día del rango, a las 00:00 UTC (convención estándar de efemérides):
 - Longitud de cada planeta (grado, signo, minuto).
 - Velocidad o indicador de retrogradación.
 - Declinación (opcional v1).
 - Longitud de la Luna a 00:00 y 12:00 (por su velocidad rápida).

3. Arquitectura Swift

3.1 Ubicación en el Proyecto

```

\ \ \
Sources/AstroMalik/
├── Engine/
│   └── Ephemeris/
│       ├── EphemerisEngine.swift      ← Motor principal de
eventos celestes
│       ├── LunationCalculator.swift    ← Lunaciones + fases
│       └── EclipseCalculator.swift     ← Eclipses solares y
lunares
│       ├── StationCalculator.swift     ← Estaciones planetarias
│       ├── SignIngressCalculator.swift ← Ingresos en signo
│       ├── VoidOfCourseCalculator.swift ← Luna vacía de curso
│       └── MundaneAspectCalculator.swift ← Aspectos entre planetas
en tránsito
├── Models/
│   └── EphemerisEvent.swift           ← Modelos de datos
├── Views/
│   └── EphemerisCalendarView.swift    ← Vista principal
calendario mensual
│   ├── EphemerisDayDetailView.swift   ← Detalle del día
│   └── EphemerisTableView.swift       ← Tabla de efemérides
clásica
\ \ \
```

3.2 Principios de Diseño

1. ****Calculadores puros**** (`enum` con funciones estáticas, sin estado, testables). Cada calculador resuelve un tipo de evento celeste y devuelve un array de `EphemerisEvent`.
2. ****Motor orquestador**** (`EphemerisEngine`) que recibe un rango de fechas, invoca todos los calculadores y devuelve un `EphemerisMonth` o `EphemerisRange` consolidado.
3. ****Sin dependencia de carta natal.**** El calendario funciona sin carta guardada. Si hay una carta activa, se puede ofrecer una capa opcional de "aspectos a natal", pero eso es un futuro sprint, no v1.
4. ****Cancelación con `Task.checkCancellation()`**** en los loops largos, mismo patrón que `TransitEngine`.
5. ****Misma infraestructura de Swiss Ephemeris****: `CSwissEph`, `swe\_calc\_ut`, `swe\_julday`, `SEFLG\_SPEED`. Sin nuevas dependencias.

3.3 Modelos de Datos

```

\ \ \ swift
// MARK: - EphemerisEvent.swift

/// Tipo unificado de evento celeste para el calendario.
enum CelestialEventKind: String, Codable, CaseIterable {
    case newMoon           // Luna Nueva
```

```

    case fullMoon           // Luna Llena
    case firstQuarter       // Cuarto Creciente
    case lastQuarter        // Cuarto Menguante
    case solarEclipse       // Eclipse solar
    case lunarEclipse       // Eclipse lunar
    case stationRetrograde  // Estación retrógrada
    case stationDirect      // Estación directa
    case signIngress        // Ingreso en signo
    case voidOfCourse       // Luna vacía de curso (inicio)
    case voidOfCourseEnd    // Luna vacía de curso (fin = ingreso
lunar)
    case mundaneAspect     // Aspecto entre planetas
}

/// Evento celeste individual.
struct CelestialEvent: Identifiable, Codable, Equatable {
    let id: UUID
    let kind: CelestialEventKind
    let dateUTC: String      // ISO "2026-06-15T14:32Z"
    let dateLocal: String    // "2026-06-15 16:32" (según
timezone del usuario)
    let longitude: Double?   // Grado eclíptico del evento
(ej: grado de la Luna Nueva)
    let signKey: String?     // "CANCER", "LEO"...
    let signLabel: String?   // "♋ Cáncer"
    let formatted: String?   // "♋ Cáncer 24°18'"

    // Datos específicos por tipo
    let planetKeyA: String?  // Planeta principal o primero
del aspecto
    let planetLabelA: String?
    let planetKeyB: String?  // Segundo planeta (para
aspectos)
    let planetLabelB: String?
    let aspectKey: String?   // "CONJUNCION", "TRIGONO"...
    let aspectLabel: String?

    // Eclipses
    let eclipseType: String? // "total", "anular", "parcial",
"penumbral"
    let eclipseMagnitude: Double?

    // Estaciones
    let stationSpeed: Double? // Velocidad en el momento (≈0)

    // VoC
    let voidEnds: String?     // ISO datetime de fin del VoC
    let voidDurationMinutes: Int?
    let lastAspectPlanet: String?
    let lastAspectType: String?

    // Ingreso
    let ingressDirection: String? // "directo" o "retrógrado"

```

```

    // Resumen para UI
    let title: String           // "🌑 Luna Nueva en ♋ Cáncer"
    let subtitle: String?      // "24°18' – 15 jun 2026 16:32"
    let importance: EventImportance
}

```

```

enum EventImportance: Int, Codable, Comparable {
    case minor = 1           // Aspecto mundano menor, ingreso lunar
    case moderate = 2        // Ingreso planetario, aspecto mundano
    case mayor, VoC
    case major = 3           // Lunación, estación planetaria
    case critical = 4        // Eclipse

    static func < (lhs: Self, rhs: Self) -> Bool {
        lhs.rawValue < rhs.rawValue
    }
}

```

```

/// Posición diaria para la tabla de efemérides.
struct DailyEphemerisRow: Identifiable, Codable {
    let id: UUID
    let date: String          // ISO "2026-06-15"
    let positions: [PlanetDailyPosition]
    let lunarPhaseAngle: Double // 0-360 (para icono de fase)
    let lunarPhaseLabel: String // "Creciente", "Llena"...
}

```

```

struct PlanetDailyPosition: Codable, Equatable {
    let planetKey: String
    let longitude: Double
    let formatted: String      // "♋ 24°18'"
    let speed: Double
    let retrograde: Bool
    let signKey: String
}

```

```

/// Contenedor del calendario mensual.
struct EphemerisMonth: Identifiable {
    let id: String             // "2026-06"
    let year: Int
    let month: Int
    let events: [CelestialEvent]
    let dailyRows: [DailyEphemerisRow]
}
```

```

### ### 3.4 Motor – EphemerisEngine

```

```swift
// MARK: – EphemerisEngine.swift

```

```

enum EphemerisEngine {

    /// Calcula todos los eventos celestes para un rango de fechas.

```

```

    /// Es la función principal que orquesta los calculadores.
    static func computeMonth(
        year: Int,
        month: Int,
        timezone: String
    ) async throws -> EphemerisMonth {
        let (startJD, endJD) = jdRangeForMonth(year: year, month:
month)

        async let lunations = LutationCalculator.findLunations(from:
startJD, to: endJD, timezone: timezone)
        async let eclipses = EclipseCalculator.findEclipses(from:
startJD, to: endJD, timezone: timezone)
        async let stations = StationCalculator.findStations(from:
startJD, to: endJD, timezone: timezone)
        async let ingresses =
SignIngressCalculator.findIngresses(from: startJD, to: endJD,
timezone: timezone)
        async let voids =
VoidOfCourseCalculator.findVoidPeriods(from: startJD, to: endJD,
timezone: timezone)
        async let aspects =
MundaneAspectCalculator.findAspects(from: startJD, to: endJD,
timezone: timezone)
        async let daily = computeDailyRows(from: startJD, to: endJD,
timezone: timezone)

        let allEvents = try await (
            lunations + eclipses + stations + ingresses + voids +
aspects
        ).sorted { $0.dateUTC < $1.dateUTC }

        return EphemerisMonth(
            id: String(format: "%04d-%02d", year, month),
            year: year,
            month: month,
            events: allEvents,
            dailyRows: try await daily
        )
    }
}

```

/// Efeméride diaria: posiciones a 00:00 UTC de cada día del mes.

```

    static func computeDailyRows(
        from startJD: Double,
        to endJD: Double,
        timezone: String
    ) async throws -> [DailyEphemerisRow] {
        // Para cada día: calcPlanets a mediodía UTC, formatear
posiciones
        // ...
    }

    static func jdRangeForMonth(year: Int, month: Int) -> (Double,

```

```

Double) {
    let startJD = swe_julday(Int32(year), Int32(month), 1, 0,
SE_GREG_CAL)
    let nextMonth = month == 12 ? 1 : month + 1
    let nextYear = month == 12 ? year + 1 : year
    let endJD = swe_julday(Int32(nextYear), Int32(nextMonth), 1,
0, SE_GREG_CAL)
    return (startJD, endJD)
}
}
...

```

3.5 Calculadores Individuales – Contratos

LurationCalculator

```

```swift
enum LurationCalculator {
 /// Encuentra Lunas Nuevas y Llenas en el rango.
 /// Método: calcular diferencia angular Sol-Luna cada 6h,
 /// detectar cruces de 0° (nueva) y 180° (llena),
 /// refinar por bisección a ±1 min.
 static func findLunations(
 from startJD: Double,
 to endJD: Double,
 timezone: String
) async throws -> [CelestialEvent]

 /// Cuartos creciente y menguante (cruces de 90° y 270°).
 /// Mismo método.
 static func findQuarters(
 from startJD: Double,
 to endJD: Double,
 timezone: String
) async throws -> [CelestialEvent]

 /// Fase lunar para un JD dado.
 static func lunarPhase(at jd: Double) throws -> (angle: Double,
label: String)
}
...

```

#### #### EclipseCalculator

```

```swift
enum EclipseCalculator {
    /// Usa swe_sol_eclipse_when_glob y swe_lun_eclipse_when en
bucle
    /// para encontrar todos los eclipses del rango.
    static func findEclipses(
        from startJD: Double,
        to endJD: Double,
        timezone: String
    ) async throws -> [CelestialEvent]
}

```

```
}  
...
```

StationCalculator

```
```swift  
enum StationCalculator {
 /// Para Mercurio-Plutón: detecta cruces de velocidad=0
 /// con muestreo diario + bisección.
 /// Planetas: Mercurio, Venus, Marte, Júpiter, Saturno, Urano,
 Neptuno, Plutón.
 static func findStations(
 from startJD: Double,
 to endJD: Double,
 timezone: String
) async throws -> [CelestialEvent]
}
...
```

#### #### SignIngressCalculator

```
```swift  
enum SignIngressCalculator {  
    /// Detecta cruces de múltiplos de 30° en la longitud de cada  
    planeta.  
    /// Incluye ingresos directos y retrógrados.  
    /// Planetas: Sol a Plutón. Luna solo si se pide explícitamente.  
    static func findIngresses(  
        from startJD: Double,  
        to endJD: Double,  
        timezone: String,  
        includeMoon: Bool = false  
    ) async throws -> [CelestialEvent]  
}  
...
```

VoidOfCourseCalculator

```
```swift  
enum VoidOfCourseCalculator {
 /// Para cada ingreso lunar del rango:
 /// 1. Encontrar el momento del ingreso (Luna cruza 30°
 boundary).
 /// 2. Retroceder y encontrar el último aspecto ptolemaico
 exacto
 /// de la Luna con Sol-Plutón antes de ese ingreso.
 /// 3. El VoC va desde ese último aspecto hasta el ingreso.
 ///
 /// Genera dos eventos por VoC: inicio (kind: .voidOfCourse)
 /// y fin (kind: .voidOfCourseEnd).
 static func findVoidPeriods(
 from startJD: Double,
 to endJD: Double,
 timezone: String
```

```

) async throws -> [CelestialEvent]
}
...

```

#### #### MundaneAspectCalculator

```

```swift
enum MundaneAspectCalculator {
    /// Aspectos entre planetas (no Luna en v1 mensual).
    /// Para cada par de planetas (45 pares de 10 planetas),
    /// detectar momentos exactos de los 5 aspectos ptolemaicos.
    ///
    /// Filtro de relevancia: solo aspectos entre al menos un
planeta
    /// lento (Júpiter-Plutón) o entre Marte y exteriores.
    /// Aspectos Sol-planetas incluidos.
    /// Aspectos Luna-planetas: solo en vista diaria.
    static func findAspects(
        from startJD: Double,
        to endJD: Double,
        timezone: String,
        includeLunar: Bool = false
    ) async throws -> [CelestialEvent]
}
...

```

3.6 Integración con la App

Navegación

Añadir a `AppNavigation.swift`:

```

```swift
// En NavItem:
case efemerides = "Efemérides"

// En systemImage:
case .efemerides: return "calendar.day.timeline.leading"

// En DetailRoute:
case ephemeris

// En ContentView detailView:
case .ephemeris:
 EphemerisCalendarView()
 .environmentObject(appState)
...

```

**\*\*Posición en la sidebar:\*\*** Después de "Tránsitos" y antes de "Horaria". Conceptualmente: Tránsitos personaliza el cielo respecto a una carta; Efemérides muestra el cielo en bruto. Horaria es consulta puntual.

#### #### Vista Principal – EphemerisCalendarView

La vista es un calendario mensual interactivo con tres capas:

1. **Cabecera con navegación de meses** (< Junio 2026 >).
2. **Grid mensual** tipo calendario con:
  - Número del día.
  - Icono de fase lunar (●●●●●●●●).
  - Indicadores compactos de eventos relevantes: emoji/icono por tipo.
  - Color de fondo según importancia del día.
3. **Lista de eventos del día seleccionado** (panel inferior o lateral según espacio).
4. **Tab "Efemérides"** con tabla clásica de posiciones diarias.

#### Nota Joplin

``EphemerisNoteBuilder`` genera una nota Markdown con:

- Mes/año.
- Lista de eventos del mes ordenados cronológicamente.
- Mini-tabla de efemérides si el usuario lo pide.

### ### 3.7 Patrón de Bisección Reutilizable

Muchos calculadores necesitan encontrar el JD exacto donde una función angular cruza un valor objetivo. Extraer esto como utilidad:

```
```swift
/// Busca por bisección el JD donde `angularFunction(jd)` cruza
/// `target`.
/// `angularFunction` devuelve un ángulo 0-360.
/// `startJD` y `endJD` deben acotar un cruce (el signo de
/// `f(start)-target`
/// debe ser opuesto al de `f(end)-target` en espacio circular).
/// Precisión: se detiene cuando |endJD - startJD| < toleranceJD.
func bisectAngularCrossing(
    startJD: Double,
    endJD: Double,
    target: Double,
    toleranceJD: Double = 1.0 / 1440.0, // ~1 minuto
    angularFunction: (Double) throws -> Double
) rethrows -> Double
```
```

Esta función se usa en:

- ``LunationCalculator``: buscar dónde ``angDiff(Sol, Luna) == 0`` o ``== 180``.
- ``SignIngressCalculator``: buscar dónde ``longitude(P) == N*30``.
- ``StationCalculator``: buscar dónde ``speed(P) == 0``.
- ``VoidOfCourseCalculator``: buscar dónde el aspecto Luna-planeta es exacto.

---

## ## 4. Plan de Tests

```

 ...
Tests/AstroMalikTests/
└─ EphemerisTests/
 └─ LurationCalculatorTests.swift
 └─ EclipseCalculatorTests.swift
 └─ StationCalculatorTests.swift
 └─ SignIngressCalculatorTests.swift
 └─ VoidOfCourseCalculatorTests.swift
 └─ MundaneAspectCalculatorTests.swift
 ...

```

**\*\*Fixtures de referencia\*\*** (verificables con cualquier efeméride online):

```

Evento	Fecha esperada (UTC aprox.)	Verificación
Luna Nueva junio 2026	~2026-06-12	Confirmar con Astro.com
Luna Llena junio 2026	~2026-06-26	Confirmar con Astro.com
Eclipse (buscar en 2026)	Verificar tipo y fecha	
swe_sol_eclipse_when_glob		
Mercurio estación retrógrada 2026	~3 estaciones en el año	
Verificar con efemérides		
Saturno ingreso Aries 2026	Verificar fecha exacta	Evento mayor
del año		
VoC: duración > 0 y < 48h	Sanity check	Ningún VoC dura más de
2.5 días |

```

---

## ## 5. Fases de Implementación

```

Sprint 1 – Motor base y lunaciones (estimación: 1 sesión Codex)
- `bisectAngularCrossing` utilidad.
- `LurationCalculator` con lunaciones y cuartos.
- `lunarPhase()` para fase diaria.
- Modelo `CelestialEvent`.
- Tests de lunaciones contra fechas conocidas.

```

```

Sprint 2 – Eclipses y estaciones (estimación: 1 sesión Codex)
- `EclipseCalculator` con `swe_sol_eclipse_when_glob` y
`swe_lun_eclipse_when`.
- `StationCalculator` con detección de velocidad cero.
- Tests.

```

```

Sprint 3 – Ingresos, VoC y aspectos mundanos (estimación: 1
sesión Codex)
- `SignIngressCalculator`.
- `VoidOfCourseCalculator`.
- `MundaneAspectCalculator`.
- Tests.

```

```

Sprint 4 – Orquestador y efeméride (estimación: 1 sesión Codex)
- `EphemerisEngine` con `computeMonth`.

```

- `DailyEphemerisRow` para tabla.
- Test de integración del mes completo.

### Sprint 5 – UI calendario (estimación: 1-2 sesiones Codex)

- `EphemerisCalendarView` con grid mensual.
- `EphemerisDayDetailView`.
- `EphemerisTableView`.
- Integración en `AppNavigation`, `ContentView`.
- `EphemerisNoteBuilder` para Joplin.
- `scripts/package\_app.sh`.

---

## ## 6. Prompt Maestro para Codex

```text

Trabajas en el repo AstroMalik-macOS. Antes de editar, inspecciona los modelos, motores, vistas y tests existentes relacionados con la funcionalidad. No generes código aislado: integra la feature en la arquitectura actual de SwiftUI, AstroEngine, modelos Codable/Equatable y stores existentes cuando aplique.

Respetar estas reglas del repo:

- Swift 6, macOS 14+, SwiftUI, SPM, sin dependencias externas salvo que se justifique.
- CSwissEph es el wrapper C de Swiss Ephemeris (target C en Sources/CSwissEph).
- No usar force unwraps.
- Mantener UI de ventana única con NavigationSplitView.
- Añadir tests enfocados para el motor nuevo y para cualquier contrato de datos.
- Si hay cambios de código o UI, ejecutar `swift test` cuando sea viable y después `scripts/package\_app.sh`.
- Antes de cerrar, comprobar el timestamp de `AstroMalik.app/Contents/MacOS/AstroMalik`.
- Actualizar docs/ARCHITECTURE.md si se añade una sección funcional nueva.
- Utilizar siempre Joplin como notas. Está en local con Web Clipper (127.0.0.1:41184).

Entrega:

- Lista de archivos modificados.
 - Resumen de decisiones.
 - Pruebas ejecutadas.
 - Limitaciones doctrinales o técnicas que queden documentadas.
- ```

7. Prompt Detallado por Sprint

Sprint 1 – Lunaciones y Utilidad de Bisección

```text

Implementa el cálculo de lunaciones (Lunas Nuevas, Llenas, Cuartos) para el módulo de Calendario Astrológico de AstroMalik.

Contexto:

- La app tiene AstroEngine (Sources/AstroMalik/Engine/AstroEngine.swift) con swe\_calc\_ut, SEFLG\_SPEED, calcPlanets() que devuelve [String: RawPlanet].
- JulianDay.swift convierte fecha local a JD UT.
- Los planetas se calculan con PLANET\_LIST usando SE\_SUN, SE\_MOON, etc.
- TransitEngine ya demuestra el patrón de loop diario con swe\_julday, swe\_calc\_ut y cancelación Task.checkCancellation().
- El wrapper CSwissEph expone toda la API de swephexp.h, incluyendo swe\_mooncross\_ut, swe\_sol\_eclipse\_when\_glob, swe\_lun\_eclipse\_when, swe\_rise\_trans, swe\_pheno\_ut.

Tareas:

1. Crear una utilidad de bisección genérica en Sources/AstroMalik/Engine/Ephemeris/AngularBisection.swift:
  - Función ``bisectAngularCrossing(startJD:endJD:target:toleranceJD:angularFunction:)`` que busca el JD donde una función angular cruza un valor objetivo.
    - Tolerancia por defecto: 1/1440 de día (~1 minuto).
    - Manejar correctamente la circularidad ( $0^\circ/360^\circ$ ).
2. Crear Sources/AstroMalik/Engine/Ephemeris/LunationCalculator.swift:
  - ``enum LunationCalculator`` con funciones estáticas.
  - ``findLunations(from:to:timezone:) async throws -> [CelestialEvent]``:
    - \* Muestrear la diferencia angular Sol-Luna cada 6 horas.
    - \* Detectar cruces de  $0^\circ$  (Luna Nueva) y  $180^\circ$  (Luna Llena).
    - \* Refinar cada cruce con `bisectAngularCrossing`.
    - \* Para cada lunación, calcular longitud eclíptica, signo y formato.
  - ``findQuarters(from:to:timezone:) async throws -> [CelestialEvent]``:
    - \* Lo mismo para cruces de  $90^\circ$  y  $270^\circ$  (cuartos).
  - ``lunarPhase(at jd:) throws -> (angle: Double, label: String)``:
    - \* Calcular el ángulo Sol-Luna y devolver la fase (8 fases).
3. Crear Sources/AstroMalik/Models/EphemerisEvent.swift con:
  - ``CelestialEventKind`` enum con todos los casos del diseño: `newMoon`, `fullMoon`, `firstQuarter`, `lastQuarter`, `solarEclipse`, `lunarEclipse`, `stationRetrograde`, `stationDirect`, `signIngress`, `voidOfCourse`, `voidOfCourseEnd`, `mundaneAspect`.
  - ``CelestialEvent`` struct: Identifiable, Codable, Equatable.

Campos: id (UUID), kind, dateUTC (ISO), dateLocal, longitude, signKey, signLabel, formatted, planetKeyA, planetLabelA, planetKeyB, planetLabelB, aspectKey, aspectLabel, eclipseType, eclipseMagnitude, stationSpeed, voidEnds, voidDurationMinutes, lastAspectPlanet, lastAspectType, ingressDirection, title, subtitle, importance.

- `EventImportance` enum: minor(1), moderate(2), major(3), critical(4).
- `DailyEphemerisRow` struct para la tabla de efemérides (puede quedar como placeholder para Sprint 4).

#### 4. Tests en Tests/AstroMalikTests/EphemerisTests/LunationCalculatorTests.swift:

- Verificar que findLunations para junio 2026 encuentra exactamente 1 Luna Nueva y 1 Luna Llena (o 2 si el mes tiene dos).
- Verificar que la Luna Nueva cae en el signo correcto (comparar con efemérides conocidas).
- Verificar que lunarPhase(at:) devuelve "Nueva" para un JD de Luna Nueva conocida y "Llena" para un JD de Luna Llena conocida.
- Verificar que findQuarters para un mes dado devuelve 2 cuartos.
- Test de bisectAngularCrossing con función trivial (lineal).

#### 5. Formateo de fechas: reutilizar el patrón de TransitEngine para ISO

y añadir un formateador que convierta JD a string local con timezone.

Reutilizar el patrón de SolarReturnEngine.formatJD si es apropiado.

No crear UI todavía. Solo motor y tests.

```

Sprint 2 – Eclipses y Estaciones

```text

Implementa el cálculo de eclipses y estaciones planetarias para el módulo de Calendario Astrológico de AstroMalik.

Contexto:

- Sprint 1 ya creó LunationCalculator, bisectAngularCrossing, CelestialEvent y EphemerisEvent.swift.
- CSwissEph (swephexp.h) expone:
  - \* swe\_sol\_eclipse\_when\_glob(tjd\_start, ifl, ifltype, tret, backward, serr)
    - busca siguiente eclipse solar global. tret[0]=máximo, tret[1]=inicio, tret[2]=fin. Return flags indican tipo (SE\_ECL\_TOTAL, SE\_ECL\_ANNULAR, SE\_ECL\_PARTIAL).

```

 * swe_lun_eclipse_when(tjd_start, ifl, ifltype, tret, backward,
serr)
 → busca siguiente eclipse lunar global. Return flags:
SE_ECL_TOTAL,
 SE_ECL_PARTIAL, SE_ECL_PENUMBRAL.
 * swe_sol_eclipse_how(tjd, ifl, geopos, attr, serr)
 → atributos del eclipse solar para un lugar. attr[0]=magnitud.
 * swe_lun_eclipse_how(tjd, ifl, geopos, attr, serr)
 → atributos del eclipse lunar. attr[0]=magnitud umbral.
 * Los flags de tipo eclipse están definidos en swephexp.h como
macros.

```

#### Tareas:

##### 1. Crear Sources/AstroMalik/Engine/Ephemeris/

###### EclipseCalculator.swift:

```

- `enum EclipseCalculator`.
- `findEclipses(from:to:timezone:) async throws ->
[CelestialEvent]`:
 * Bucle con swe_sol_eclipse_when_glob desde startJD, avanzando
 tras cada eclipse encontrado, hasta superar endJD.
 * Bucle con swe_lun_eclipse_when desde startJD, mismo patrón.
 * Para cada eclipse, calcular longitud del Sol (solar) o Luna
(lunar)
 en el JD del máximo con swe_calc_ut.
 * Determinar tipo (total/anular/parcial/penumbral) desde los
flags.
 * Magnitud con swe_sol_eclipse_how o swe_lun_eclipse_how.
 * importance = .critical.
 * title = "● Eclipse Solar Total en ♌ Leo" o similar.

```

##### 2. Crear Sources/AstroMalik/Engine/Ephemeris/

###### StationCalculator.swift:

```

- `enum StationCalculator`.
- `findStations(from:to:timezone:) async throws ->
[CelestialEvent]`:
 * Para cada planeta en [SE_MERCURY, SE_VENUS, SE_MARS,
SE_JUPITER,
 SE_SATURN, SE_URANUS, SE_NEPTUNE, SE_PLUTO]:
 - Calcular velocidad eclíptica (xx[3]) cada día del rango.
 - Detectar cambios de signo en la velocidad.
 - Refinar por bisección el JD donde speed≈0
 (usar bisectAngularCrossing adaptado o función similar
 que busque cruce de cero en función escalar).
 - Determinar tipo: velocidad positiva→negativa = SR,
 negativa→positiva = SD.
 * importance = .major.
 * title = "♄ Saturno estación retrógrada en ♈ Aries 02°15'"

```

##### 3. Crear utilidad auxiliar si es necesario: `bisectScalarCrossing` para buscar cruce de cero en una función escalar (velocidad), distinto de bisectAngularCrossing que maneja circularidad.

##### 4. Tests:

```

- EclipseCalculatorTests: para el año 2026, verificar que se

```

- encuentran eclipses (al menos hay eclipses solares y lunares cada año). Verificar tipo y fecha aproximada contra efemérides.
- StationCalculatorTests: para 2026, verificar que Mercurio tiene exactamente 3 pares de estaciones SR+SD. Verificar que Saturno tiene exactamente 1 par SR+SD. Verificar que la velocidad en el JD de la estación es  $< 0.01^\circ/\text{día}$  en valor absoluto.

No crear UI todavía.

```

Sprint 3 – Ingresos, VoC y Aspectos Mundanos

```text

Implementa ingresos en signo, Luna vacía de curso y aspectos mundanos para el módulo de Calendario Astrológico de AstroMalik.

Contexto:

- Sprints anteriores ya crearon LunationCalculator, EclipseCalculator, StationCalculator, bisectAngularCrossing, CelestialEvent.
- AstroEngine.calcPlanets(jd:) devuelve [String: RawPlanet] con key, deg, speed, retro.
- PLANET\_LIST tiene los 10 planetas principales.
- ASPECT\_DEFS define los 5 aspectos ptolemaicos con ángulos.

Tareas:

#### 1. Crear Sources/AstroMalik/Engine/Ephemeris/

SignIngressCalculator.swift:

- Para cada planeta (Sol a Plutón, Luna opcional con flag):
  - \* Calcular longitud diaria.
  - \* Detectar cruces de múltiplos de  $30^\circ$ :  
si  $\text{floor}(\text{lon\_hoy}/30) \neq \text{floor}(\text{lon\_ayer}/30)$ , hay ingreso.
  - \* Cuidar el caso de retrogradación: planeta puede cruzar  $30^\circ$  hacia atrás (ingreso retrógrado al signo anterior).
  - \* Refinar JD exacto con bisección.
  - \* Marcar si el ingreso es directo o retrógrado.
- importance:
  - \* Sol: .moderate (define estaciones del año).
  - \* Júpiter-Plutón: .moderate (ingresos de años/décadas).
  - \* Mercurio-Marte: .minor.
  - \* Luna: .minor (solo si includeMoon=true).

#### 2. Crear Sources/AstroMalik/Engine/Ephemeris/

VoidOfCourseCalculator.swift:

- findVoidPeriods(from:to:timezone:):
  - \* Primer paso: encontrar todos los ingresos lunares en el rango (reutilizar SignIngressCalculator con includeMoon=true, filtrando solo Luna, o calcular directamente).
  - \* Para cada ingreso lunar:
    - Retroceder desde el JD del ingreso.
    - Para cada planeta (Sol a Plutón, 9 planetas):  
buscar el último aspecto ptolemaico exacto de la Luna

- con ese planeta antes del ingreso.
- El VoC empieza en el más reciente de esos últimos aspectos.
- Si no se encuentra ningún aspecto en las últimas 72h, marcar como VoC extendido (raro pero posible).
- \* Generar dos eventos: inicio (.voidOfCourse) y fin (.voidOfCourseEnd).
- \* importance = .moderate.
- Aspecto exacto de Luna: usar bisección para encontrar el JD donde  
 $\text{angDiff}(\text{Luna}, \text{Planeta}) == \text{ángulo del aspecto}.$   
 Buscar hacia atrás desde el ingreso, con step de 2h (la Luna se mueve  $\sim 0.5^\circ/\text{h}$ , así que 2h es suficiente resolución).

### 3. Crear Sources/AstroMalik/Engine/Ephemeris/MundaneAspectCalculator.swift:

- findAspects(from:to:timezone:includeLunar:):
- \* Pares de planetas relevantes: filtrar para evitar ruido excesivo.
- Regla: al menos uno de los dos planetas debe ser lento (Marte a Plutón) o ser el Sol.
- No incluir pares Luna-planeta en vista mensual (demasiados).
- No incluir pares Mercurio-Venus ni Venus-Mercurio solos.
- \* Para cada par, calcular la diferencia angular diaria.
- \* Detectar cruces de los 5 ángulos ptolemaicos (0, 60, 90, 120, 180).
- \* Refinar por bisección.
- \* importance:
  - .major si involucra dos lentos (Júpiter-Plutón entre sí).
  - .moderate si involucra Sol con lento o Marte con lento.
  - .minor para el resto.
- \* Clasificar calidad: trígono/sextil = benéfico, cuadratura/oposición = tenso, conjunción = depende de planetas.

### 4. Tests:

- SignIngressCalculator: verificar que el Sol ingresa en Cáncer  $\sim 21$  junio 2026. Verificar que detecta ingresos retrógrados (buscar un planeta lento que retrograda en 2026).
- VoidOfCourseCalculator: para una semana concreta, verificar que cada VoC tiene duración  $> 0$  y  $< 48\text{h}$ . Verificar que el VoC termina exactamente en un ingreso lunar.
- MundaneAspectCalculator: para un mes, verificar que encuentra al menos algunos aspectos. Verificar que Saturno-Urano, si forman aspecto en el rango, aparece como .major.

No crear UI todavía.

```

Sprint 4 – Orquestador y Efeméride Diaria

```text

Implementa el motor orquestador EphemerisEngine y la tabla de efemérides

diaria para el módulo de Calendario Astrológico de AstroMalik.

Contexto:

- Sprints 1-3 crearon todos los calculadores individuales: LurationCalculator, EclipseCalculator, StationCalculator, SignIngressCalculator, VoidOfCourseCalculator, MundaneAspectCalculator.
- Todos devuelven [CelestialEvent].
- El modelo DailyEphemerisRow ya existe como placeholder.

Tareas:

1. Crear Sources/AstroMalik/Engine/Ephemeris/EphemerisEngine.swift:

- `enum EphemerisEngine`.
- `computeMonth(year:month:timezone:) async throws ->

EphemerisMonth`:

- \* Calcular JD de inicio y fin del mes.
- \* Invocar TODOS los calculadores con async let para paralelismo.
- \* Consolidar en array único ordenado por dateUTC.
- \* Calcular las filas diarias con computeDailyRows.
- \* Devolver EphemerisMonth.
- `computeDailyRows(from:to:timezone:) throws -> [DailyEphemerisRow]`:
- \* Para cada día del rango, a las 00:00 UTC:
  - Calcular posiciones de los 10 planetas + Nodo Norte con AstroEngine.calcPlanets y AstroEngine.calcLunarNodes.
  - Para cada planeta: longitud, formatted (grado signo minuto), speed, retrograde, signKey.
  - Fase lunar con LurationCalculator.lunarPhase(at:).
- \* Devolver array de DailyEphemerisRow.

2. Completar el modelo EphemerisMonth en EphemerisEvent.swift si falta:

- id, year, month, events, dailyRows.
- Computed property: eventsByDay -> [String: [CelestialEvent]] agrupando por fecha ISO (solo parte de fecha).

3. Tests de integración:

- EphemerisEngineTests: computeMonth para junio 2026.
  - \* Verificar que events contiene al menos: 1 Luna Nueva, 1 Luna Llena, 2 cuartos, algunos ingresos, algunos VoC, algunos aspectos mundanos.
  - \* Verificar que dailyRows tiene exactamente 30 filas (junio).
  - \* Verificar que cada dailyRow tiene 11 posiciones (10 planetas + Nodo Norte).
  - \* Verificar que el lunarPhaseAngle está en [0, 360).
- ...

### Sprint 5 – UI y Navegación

```text

Implementa la interfaz de usuario del módulo de Calendario Astrológico

de AstroMalik y su integración en la app.

Contexto:

- EphemerisEngine.computeMonth ya funciona y devuelve EphemerisMonth.
- La app usa NavigationSplitView con NavItem en sidebar.
- ContentView.swift tiene el switch de detailRoute.
- AppNavigation.swift define NavItem, DetailRoute y viewIdentity.
- El patrón de las vistas existentes: un EnvironmentObject appState, vistas con @State para datos calculados, .task {} para cálculos async.
- TransitsView demuestra el patrón de vista con cálculo async, TransitWorkspaceState para estado, y tabs.
- JoplinClipperService crea notas vía Web Clipper local.
- El tema visual sigue AppTheme.swift con Color.appBackground, Color.appPrimaryText, etc.

Tareas:

1. Añadir a AppNavigation.swift:
 - NavItem.efemerides = "Efemérides"
 - systemImage: "calendar.day.timeline.leading"
 - DetailRoute.ephemeris
 - viewIdentity: "ephemeris"
 - Posición: después de .transitos, antes de .horaria.
2. Añadir en ContentView.swift:
 - Caso .ephemeris en el switch de detailView.
 - EphemerisCalendarView().environmentObject(appState)
3. Añadir showDefaultDetail para .efemerides en AppState (inspeccionar cómo se hace para .transitos o .horaria).
4. Crear Sources/AstroMalik/Views/EphemerisCalendarView.swift:
 - @EnvironmentObject var appState: AppState
 - @State var currentYear: Int (año actual)
 - @State var currentMonth: Int (mes actual)
 - @State var ephemerisData: EphemerisMonth?
 - @State var selectedDay: Int? (día seleccionado en el grid)
 - @State var isLoading: Bool
 - @State var viewMode: EphemerisViewMode (.calendar / .table)

Layout:

- Cabecera: botón <, texto "Junio 2026", botón >, botón "Hoy", segmented picker [Calendario | Efemérides].
 - Si viewMode == .calendar:
 - * Grid de 7 columnas (L M X J V S D) con celdas por día.
 - * Cada celda: número del día, icono de fase lunar pequeño, indicadores de eventos (puntos de color o mini-iconos).
 - * Día seleccionado resaltado.
 - * Panel inferior: lista de CelestialEvent del día seleccionado, ordenados por hora. Cada evento con icono, título, hora
- local.
- Si viewMode == .table:
 - * EphemerisTableView con tabla scrollable de DailyEphemerisRow.

- * Columnas: Día | ☉ | ☿ | ♀ | ♀ | ♂ | ♄ | ♃ | ♅ | ♁ | ♁ | ♁ | ♁
- * Cada celda: grado°minuto' con símbolo de signo.
- * Planetas retrógrados marcados con R.

- Botón Joplin: genera nota del mes completo y la envía por JoplinClipperService, reutilizando el patrón de SolarReturnView.

5. Crear Sources/AstroMalik/Views/EphemerisDayDetailView.swift:

- Recibe [CelestialEvent] del día.
- Lista vertical con cada evento:
 - * Hora local.
 - * Icono/emoji por tipo.
 - * Título.
 - * Subtítulo con detalles técnicos.
- Color de fondo según importance.

6. Crear EphemerisNoteBuilder (puede ir en Views/ o en un NoteBuilder/):

- Genera Markdown con:
 - * Título: "Efemérides – Junio 2026"
 - * Sección "Eventos del mes" con lista cronológica.
 - * Sección "Eclipses" si hay.
 - * Sección "Estaciones" si hay.
 - * Sección "Lunaciones".
 - * Mini-tabla de efemérides (opcional, puede ser muy larga).

7. Ejecutar swift test y scripts/package_app.sh.
Verificar timestamp de AstroMalik.app/Contents/MacOS/AstroMalik.

8. Actualizar docs/ARCHITECTURE.md con la nueva sección "Efemérides".
```

---

## 8. Decisiones Doctrinales Explícitas

Decisión	Elección v1	Justificación
Luna VoC: ¿7 o 10 planetas?	10 (moderna)	Más conservador, VoC más cortos, menos falsos positivos
Aspectos: ¿solo ptolemaicos?	Sí (5 clásicos)	Consistente con el resto de la app
Eclipses: ¿visibilidad local?	No en v1	Complejidad alta, valor incremental
Luna en tabla diaria	Sí, a 00:00 UTC	Convención estándar de efemérides
Aspectos mundanos: ¿Luna?	No en vista mensual, sí en diaria futura	Demasiado frecuente para la vista principal
Ingresos de Luna	No por defecto	~13/mes, ruido excesivo; incluir solo en detalle diario
Orbe para aspectos mundanos	0° (exacto) + ventana ±1°	para marcar el día   El calendario marca cuándo es exacto

| Nodo en efeméride | Sí, Nodo Norte | Es dato de referencia estándar |

---

## ## 9. Futuras Extensiones (No v1)

1. **Efeméride gráfica** (gráfico con longitudes eclípticas como líneas a lo largo del mes – visual muy potente pero requiere charting).
2. **Búsqueda de eventos futuros** ("¿cuándo es la próxima conjunción Júpiter-Saturno?").
3. **Lunaciones sobre carta natal** (en qué casa cae la Luna Nueva).
4. **Eclipses sobre carta natal** (grado del eclipse vs. planetas natales).
5. **Calendario anual** (vista de 12 meses con eventos marcados).
6. **Exportación iCal** (.ics con eventos astrológicos).
7. **Aspectos de Luna en vista diaria** (15-20 aspectos lunares/día).
8. **Paralelos y contra-paralelos** (declinaciones).

---

## ## 10. Resumen de Archivos a Crear/Modificar

### ### Archivos nuevos

Archivo	Tipo
--- ---	
`Sources/AstroMalik/Engine/Ephemeris/AngularBisection.swift`	Utilidad
`Sources/AstroMalik/Engine/Ephemeris/LunationCalculator.swift`	Motor
`Sources/AstroMalik/Engine/Ephemeris/EclipseCalculator.swift`	Motor
`Sources/AstroMalik/Engine/Ephemeris/StationCalculator.swift`	Motor
`Sources/AstroMalik/Engine/Ephemeris/SignIngressCalculator.swift`	Motor
`Sources/AstroMalik/Engine/Ephemeris/VoidOfCourseCalculator.swift`	Motor
`Sources/AstroMalik/Engine/Ephemeris/MundaneAspectCalculator.swift`	Motor
`Sources/AstroMalik/Engine/Ephemeris/EphemerisEngine.swift`	Orquestador
`Sources/AstroMalik/Models/EphemerisEvent.swift`	Modelos
`Sources/AstroMalik/Views/EphemerisCalendarView.swift`	Vista principal
`Sources/AstroMalik/Views/EphemerisDayDetailView.swift`	Vista detalle
`Sources/AstroMalik/Views/EphemerisTableView.swift`	Vista tabla
`Tests/AstroMalikTests/EphemerisTests/LunationCalculatorTests.swift`	Tests
`Tests/AstroMalikTests/EphemerisTests/	

```
EclipseCalculatorTests.swift` | Tests |
| `Tests/AstroMalikTests/EphemerisTests/
StationCalculatorTests.swift` | Tests |
| `Tests/AstroMalikTests/EphemerisTests/
SignIngressCalculatorTests.swift` | Tests |
| `Tests/AstroMalikTests/EphemerisTests/
VoidOfCourseCalculatorTests.swift` | Tests |
| `Tests/AstroMalikTests/EphemerisTests/
MundaneAspectCalculatorTests.swift` | Tests |
| `Tests/AstroMalikTests/EphemerisTests/EphemerisEngineTests.swift`
| Tests |
```

### Archivos modificados

```
| Archivo | Cambio |
|---|---|
| `Sources/AstroMalik/AppNavigation.swift` | Añadir
NavItem.efemerides y DetailRoute.ephemeris |
| `Sources/AstroMalik/Views/ContentView.swift` | Añadir
caso .ephemeris en detailView |
| `Sources/AstroMalik/AstroMalikApp.swift` | Si AppState necesita
showDefaultDetail para efemérides |
| `docs/ARCHITECTURE.md` | Documentar la sección Efemérides |
```

---

\*Documento generado por Claude Opus 4.6 (Anthropic) el 4 de mayo de 2026.\*

\*Revisión completa del código fuente, docs/ARCHITECTURE.md, análisis astrológico de Claude Opus, análisis comercial y revisión arquitectónica de ChatGPT 5.5.\*